

# INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer and Communication Engineering

Course Code: CCE-3509

Course Title: Computer Architecture

---

Abu Noman Siddike  
Adjunct Faculty  
Dept. of CCE, IIUC

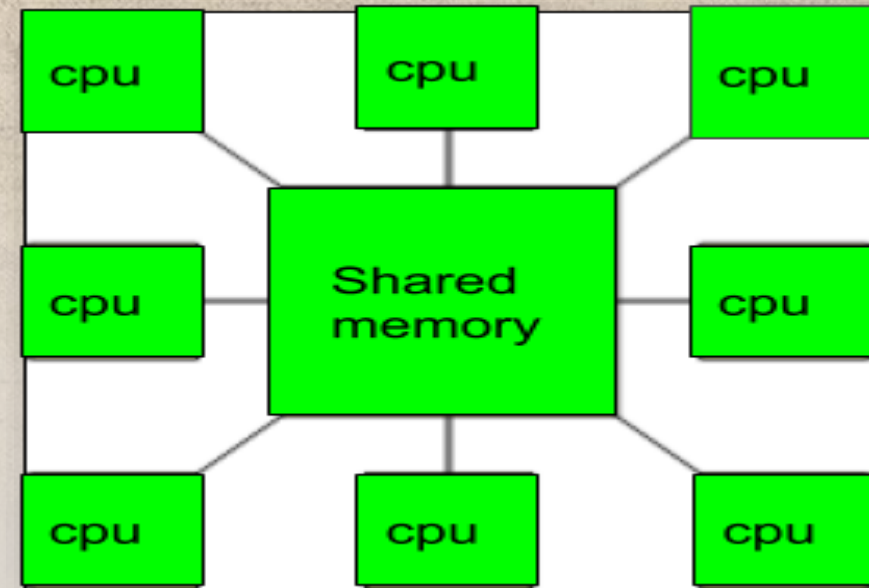


# MULTIPROCESSORS

---

A **multiprocessor system** is a computer with two or more central processing units (CPUs) that work together to execute a single program or multiple programs simultaneously. This design significantly improves system performance, throughput, and reliability.

- **Parallelism:** Multiprocessors enable parallel processing, where multiple tasks are executed at the same time.
- **Communication:** Processors communicate with each other to coordinate tasks and share data. This communication can be through a shared memory space or by passing messages.





# TYPES OF MULTIPROCESSORS

---

- i. **Symmetric Multiprocessors:** each processor contains a similar copy of the operating system and they all communicate with each other. All the processors are in a peer to peer relationship i.e. no master - slave relationship exists between them.
- ii. **Asymmetric Multiprocessors:** each processor is given a predefined task. There is a master processor that gives instruction to all the other processors. Asymmetric multiprocessor system contains a master slave relationship. Example: IBM System/370



# SYMMETRIC VS ASYMMETRIC

| Feature                  | Symmetric Multiprocessing (SMP)                                                                 | Asymmetric Multiprocessing (AMP)                                                            |
|--------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Definition</b>        | All processors are identical and share the same memory and I/O; each runs the same OS instance. | one “master” controls the system and assigns work to “slave” processors.                    |
| <b>Control</b>           | No master–slave relationship; all CPUs are peers.                                               | Master CPU schedules and controls all tasks; slaves execute assigned jobs only.             |
| <b>OS Support</b>        | Single copy of the OS manages all CPUs equally.                                                 | OS runs on the master CPU; slaves may run minimal or no OS.                                 |
| <b>Work Distribution</b> | Any CPU can run any process or thread.                                                          | Master decides which CPU runs which task.                                                   |
| <b>Fault Tolerance</b>   | If one CPU fails, others can continue (depending on OS).                                        | If master fails, the whole system may halt.                                                 |
| <b>Programming Model</b> | Easier for general-purpose workloads; OS handles load balancing.                                | Simpler hardware, but software must explicitly manage task assignment.                      |
| <b>Scalability</b>       | Scales well up to a moderate number of CPUs (bus or interconnect limits apply).                 | Less scalable for general-purpose computing; often used in embedded or specialized systems. |



# PERFORMANCE

---

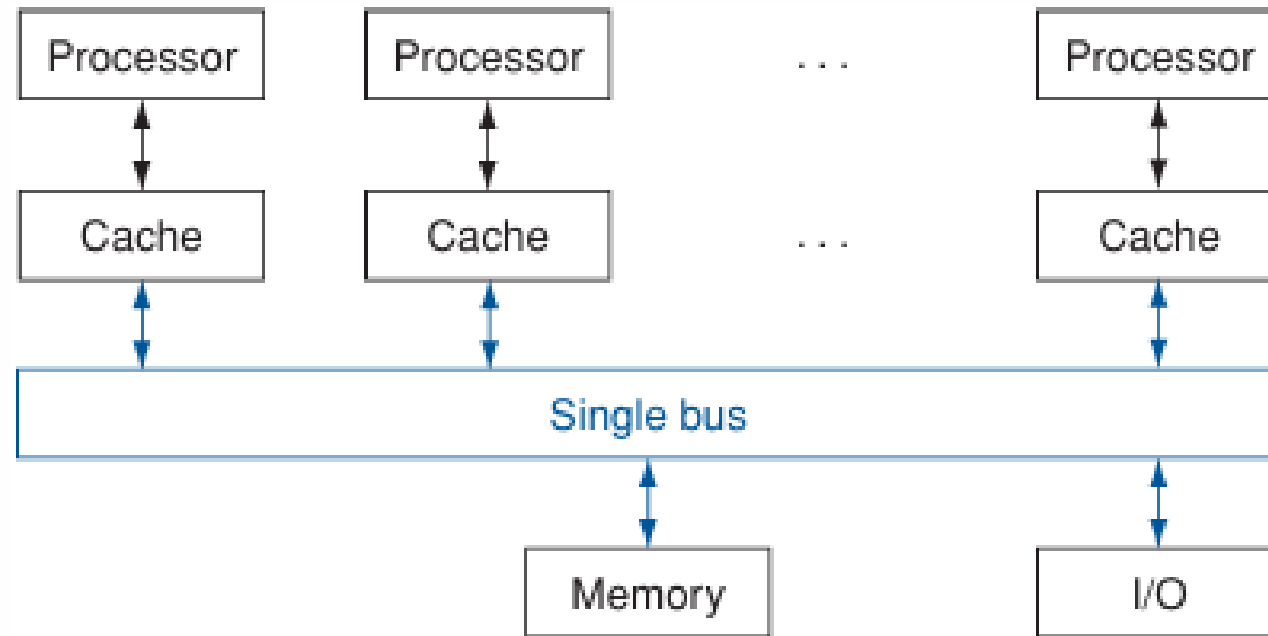
## **Benefits of using a Multiprocessor -**

- ✓ Enhanced performance.
- ✓ Multiple applications.
- ✓ Multi-tasking inside an application.
- ✓ High throughput and responsiveness.
- ✓ Hardware sharing among CPUs.



# SINGLE BUS MULTIPROCESSOR

A single-bus multiprocessor is a system where multiple processors and other components are connected via a single, shared communication path or bus for data, instructions, and control signals.





# CHARACTERISTICS

---

- **Shared Memory Access:** All CPUs see the same global address space. Memory consistency and cache coherence must be maintained.
- **Bus Arbitration:** Only one CPU/device can use the bus at a time.
- **Uniform Memory Access (UMA):** All CPUs have equal access time to any memory location.
- **Simplicity:** Easy to design and program compared to complex interconnects.

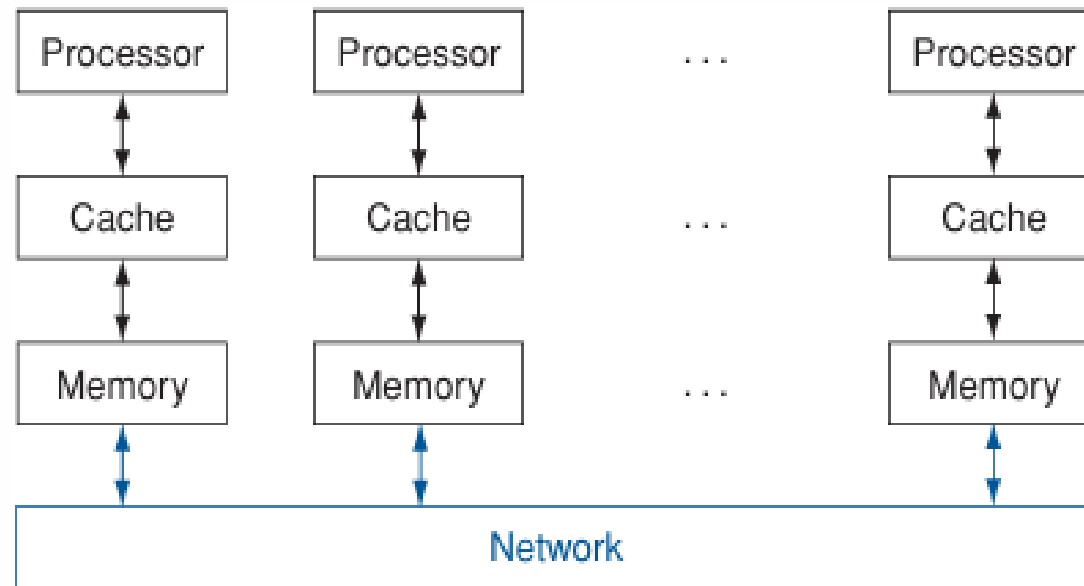
## • **Limitations:**

- **Bus Contention** – Only one transfer at a time → bottleneck as CPUs increase.
- **Poor Scalability** – Performance drops sharply beyond ~8 processors.
- **Latency** – Waiting for bus access increases memory access time.
- **Coherence Overhead** – Maintaining cache consistency adds traffic.



# MULTIPROCESSORS CONNECTED BY A NETWORK

- In this model, multiple processors (or entire computer systems) are **interconnected through a communication network** instead of sharing a single bus.
- Each processor typically has its **own private memory**, and the network provides the medium for **inter-processor communication**.





# CHARACTERISTICS

---

- **No shared memory:** Each processor has independent memory → avoids memory contention issues common in bus-based systems.
- **Message Passing:** Processors communicate by sending and receiving messages over the network.
- **Scalability:** Much easier to scale up by adding new nodes, compared to bus-based multiprocessors.
- **Fault Tolerance:** If one processor or node fails, others can continue functioning.



# CLUSTERING

---

A **cluster** is a set of loosely or tightly connected computers working together as a unified computing resource that can create the illusion of being one machine. Computer clusters have each node set to perform the same task, controlled and produced by the software.

For making cluster more efficient there exist two clusters:

- **Hardware Cluster**

- **Software Cluster**

**Hardware Cluster** helps in enable high-performance disk sharing between systems, while the **Software Cluster** allows all systems to work together.



# TYPES OF CLUSTERS

---

- i. **Asymmetric Cluster:** In this type of clustering, all the nodes run the required applications, and one node is in hot standby mode. The Hot standby node is used for monitoring the server till it fails, when it fails then it takes its place.
- ii. **Symmetric Cluster:** In this type of clustering, all the nodes run applications and monitor other nodes at the same time. This clustering is more efficient than Asymmetric clustering as it doesn't have any hot standby key.



# THE FEATURES OF CLUSTERS

---

- ✓ **High Performance** – Combine the power of multiple computers for faster computation.
- ✓ **Scalability** – Easily add more nodes to boost performance.
- ✓ **Fault Tolerance** – Continue operating despite node failures via redundancy.
- ✓ **Load Balancing** – Distribute workloads evenly to avoid bottlenecks.
- ✓ **High-Speed Interconnectivity** – Use fast networks for efficient data exchange.
- ✓ **Shared Resources** – Provide common access to storage, memory, and I/O devices.
- ✓ **Cost-Effective** – Leverage commodity hardware and open-source software.
- ✓ **Versatility** – Support diverse applications like scientific computing, analytics, and web services.

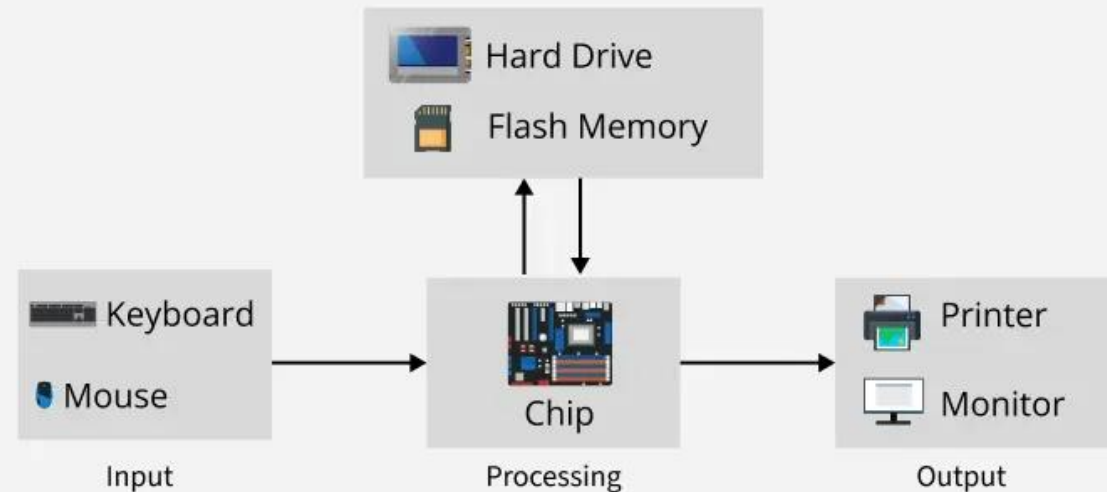


# WHAT IS COMPUTER MEMORY?

---

- A physical device that stores data or information temporarily or permanently in it is called memory.

## How does Computer Memory Works ?





# COMPUTER MEMORY HIERARCHY

---

- **Registers**

- Inside the CPU.

- Fastest storage, directly accessed by instructions.

- Very limited in number (tens to hundreds).

- **Cache Memory**

- Small, high-speed memory between CPU and main memory.

- Stores frequently accessed data.

- Levels: L1, L2, L3 (closer to CPU = faster, smaller).

- **Main Memory (RAM)**

- Stores currently executing programs and data.

- Volatile (data lost when power is off).

- Example: DRAM.

- **Secondary Storage**

- Large capacity, slower than RAM.

- Non-volatile (data retained).

- Examples: Hard Disk Drives (HDD), Solid State Drives (SSD).



# KEY CHARACTERISTICS OF MEMORY

---

- **Speed (Access Time):** How quickly data can be read/written.
- **Cost per Bit:** Registers are most expensive; disks cheapest.
- **Capacity:** Amount of data it can hold.
- **Volatility:** Whether data is retained after power is off.

## \*\*\*Why the Hierarchy Matters

CPU is much faster than main memory → without caches, CPU would idle waiting for data

Hierarchy ensures:

- Frequently used data is kept close to CPU
- Large datasets stored in cheaper, slower memory



# TYPES OF MEMORY (BY FUNCTION)

---

- **Volatile:** RAM, cache, registers.
- **Non-volatile:** ROM, SSD, HDD, flash memory.
- **Read-Only Memory (ROM):** Permanent storage of firmware.
- **Virtual Memory:** Uses part of secondary storage to act as additional RAM.



# CACHE MEMORY

---

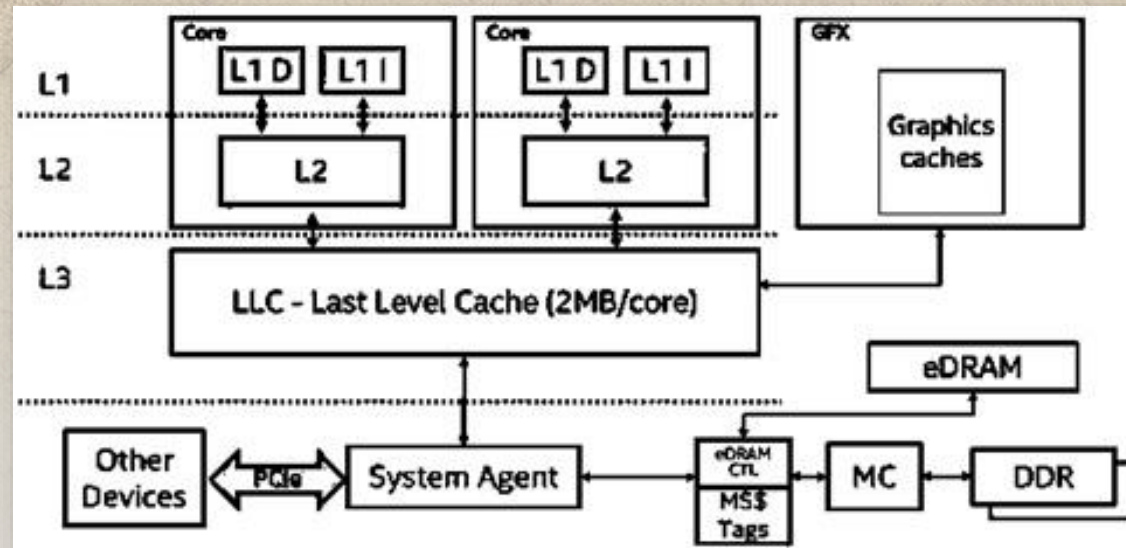
- Cache memory is a volatile memory fixed closer to the CPU to provide high-speed data access to the processor and stores frequently used computer programs, applications and data.
- It is smaller and faster than RAM is called cache memory.





# The Levels of Cache Memory: L1 Cache

- Level 1 cache is a computer's fastest memory.
- The CPU's most frequently accessed data resides in the L1 cache.
- CPU determines L1 cache size.
- Some high-end consumer CPUs, such as the Intel i9-9980XE, have a 1MB L1 cache, but they are expensive and rare.





# The Levels of Cache Memory: L2 Cache

---

- Level 2 cache is larger but slower than L1.
- Modern L2 memory caches are gigabytes, not kilobytes.
- AMD's top-rated Ryzen 5 5600X has 384KB L1 and 3MB L2 caches and 32MB L3 cache.
- The L2 cache size depends on the CPU but is usually 256KB to 32MB.
- Nowadays, most CPUs have more than 256KB L2 cache, which is small.



# The Levels of Cache Memory: L3 Cache

---

- The largest and slowest cache memory unit is L3.
- The L3 memory cache was originally on the motherboard.
- This was long ago when most CPUs were single-core.
- The L3 cache on top-end consumer CPUs can reach 32MB, while AMD's ground-breaking Ryzen 7 5800X3D CPUs have 96MB.
- CPU L3 caches in some servers can reach 128MB.



# Principles of Cache Memory

---

**1. Locality of Reference:** This is the fundamental principle behind caching.

- ❑ **Temporal Locality:** If a piece of data is accessed, it is likely to be accessed again soon.
- ❑ **Spatial Locality:** If a particular memory location is accessed, nearby memory locations will also be accessed in the near future.

**2. Speed & Proximity:** Cache memory is a small, extremely fast memory.

**3. Buffering:** The cache acts as a buffer between the CPU and the slower, larger main memory.



# Principles of Cache Memory

---

## 4.Data Fetching:

- ☐ When the CPU needs a piece of data, it first checks the cache.
- ☐ If the data is in the cache (a **cache hit**), it is delivered to the CPU quickly.
- ☐ If the data is not in the cache (a **cache miss**), the required block of data is fetched from the main memory and copied into the cache.

**5.Block Transfers:** Instead of fetching individual bytes, entire blocks or lines of data are transferred between main memory and the cache to leverage spatial locality.

**6.Performance Improvement:** By effectively storing and retrieving frequently used data, cache memory significantly reduces the average time required for the CPU to access data.



# Cache Memory Design

---

## Elements of Cache Memory Design

- **Cache Size:** It seems that moderately tiny caches will have a big impact on performance.
- **Block Size:** Block size is the unit of information changed between cache and main memory. As the block size will increase from terribly tiny to larger sizes, the hit magnitude relation can initially increase as a result of the principle of locality.
- **Cache Mapping Techniques**
- **Cache Replacement Policies**
- **Write Policies**



# Cache Memory Design

---

## **Advantages**

Cache memory speeds things up by keeping frequently used data close to the CPU. This reduces memory latency, cuts down CPU idle time, and improves instruction execution and overall system throughput.

## **Disadvantages**

Its small size limits how much data can be stored, leading to cache misses. The hardware and algorithms needed for cache management add complexity, especially in multiprocessor systems where maintaining cache coherence and data consistency is difficult and costly.



# Cache Mapping Techniques

---

Caches need to figure out *where* to put RAM data. Think of it as deciding how books are arranged on the librarian's shelf.

- **Direct Mapping:** Each memory block maps to exactly one cache line. Fast and simple, but if two frequently used blocks map to the same line, they keep evicting each other (conflict misses).
- **Fully Associative Mapping:** Any block can go into any cache line. Very flexible but requires expensive hardware for searching.
- **Set-Associative Mapping:** A compromise. Cache is divided into sets, and each block maps to a set, but within that set it can go to any line. E.g., a 4-way set-associative cache means each set holds 4 blocks. This is the most common design in modern CPUs.



# Cache Replacement Policies

---

When the cache is full and new data needs to come in, which old data gets the boot?

- **LRU (Least Recently Used):** Evict the block that hasn't been used for the longest time.
- **FIFO (First In, First Out):** Kick out the oldest block.
- **Random:** Evict randomly. Surprisingly effective in hardware due to low overhead.



# Write Policies

---

When the CPU writes data, what happens to main memory?

- **Write-Through:** Data is written to both cache and RAM at the same time. Simple but slow.
- **Write-Back:** Data is written only to the cache. RAM is updated later when the cache block is evicted (requires dirty bit). Faster, but adds complexity.



# Cache Hit and Cache Miss

---

- A **cache hit** occurs when the CPU finds the data it needs in the fast, small cache, allowing for quick retrieval.

$$\text{Hit Rate} = \frac{\text{Number of hits}}{(\text{Number of hits} + \text{Number of misses})}$$

- A **cache miss** happens when the data is not found in the cache, forcing the CPU to fetch it from the slower main memory, which significantly slows down the process.

$$\text{Miss Rate} = \frac{\text{Number of misses}}{(\text{Number of hits} + \text{Number of misses})}$$



# Performance Metrics

---

Cache efficiency is measured by:

- **Hit Rate:** Percentage of times the CPU finds data in cache. Higher is better.

$$\text{Hit Rate} = \frac{\text{Number of hits}}{(\text{Number of hits} + \text{Number of misses})}$$

- **Miss Rate:** Percentage of times the CPU must go to RAM.
- **Miss Penalty:** Time taken to fetch from RAM on a miss.

**Average Memory Access Time (AMAT):**

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} * \text{Miss Penalty})$$



# PENTIUM 4 CACHE ORGANIZATION

---

- **a) L1 Cache: Split into Instruction Cache and Data Cache.**

- **Instruction L1:** 12K  $\mu$ op Trace Cache

- Stores decoded micro-ops ( $\mu$ ops), not raw
- Avoids repeated instruction decoding.

- **Data L1:** 8 KB (later versions had 16 KB).

- 4-way set associative. 64-byte line size.
- Optimized for single-cycle access.

- **b) L2 Cache**

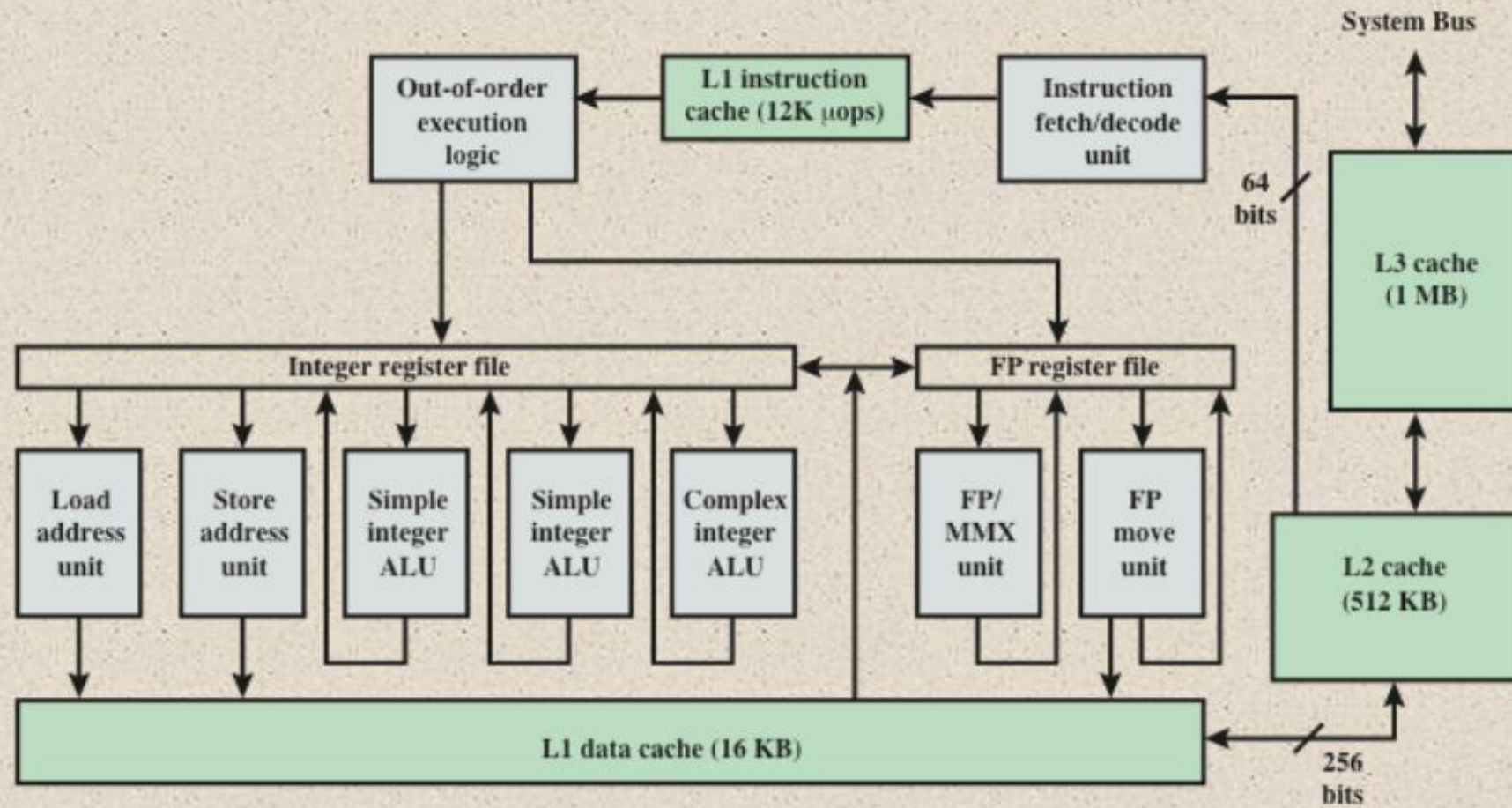
- Unified cache (stores both instructions and data).
- Size ranged from 256 KB  $\rightarrow$  512 KB  $\rightarrow$  1 MB (later 2 MB in Prescott).
- 8-way set associative.
- Latency:  $\sim$ 20 cycles.
- Exclusive design in early versions (data in L1 not duplicated in L2).

- **c) L3 Cache (later Pentium 4 models like Gallatin)**

- Introduced in some Pentium 4 Xeons.
- Size: 2 MB  $\rightarrow$  4 MB.
- Shared by all cores (for multi-core versions).
- Helped server workloads by reducing memory stalls.



# PENTIUM 4 CACHE ORGANIZATION





# ARM CACHE ORGANIZATION

---

## a) L1 Cache:

- Separate Instruction and Data caches (Harvard).
- Typical sizes: 16 KB – 64 KB each.
- Line size: 32–64 bytes.
- Configurable associativity depending on implementation.
- Focus: predictable timing and power efficiency.

## b) L2 Cache:

- Unified cache (instruction + data).
- Sizes: 128 KB – 8 MB depending on SoC.
- Often shared across cores in multi-core ARM Cortex-A series.
- Latency: higher than L1, but tuned for low power consumption.
- ARM Cortex-A processors include a “L2 Cache Controller” for handling prefetching, coherency, and power management.

## c) L3 Cache (High-End ARM, )

- Introduced in high-performance ARM cores for servers and mobile CPUs.
- Size: 1 MB – 4 MB, shared among all clusters.
- Reduces bandwidth pressure to DRAM.



